光伏电站发电功率日前预测问题

参赛编号：004178

光伏电站发电功率日前预测问题

摘　要

光伏发电是通过半导体材料的光电效应，将太阳能直接转化为电能的技术，其规模化应用对实现碳中和目标具有重要意义。开展高精度的光伏电站发电功率日前预测，有助于优化电网调度计划，提升电力系统运行安全性与经济性。本研究基于公开数据集，选取九个相邻电站 2022 年 1 月 3 日至 2023 年 1 月 3 日的 15 分钟间隔数据，基于题目要求光伏电站发电问题展开研究。

针对问题一：针对光伏电站发电特性分析问题，提出一种融合晴空模型、GWO-VMD 分解与 HMM 状态识别的多尺度分析方法。基于光伏电站**地理位置**构建**晴空模型**计算理论发电功率；通过**灰狼算法优化变分模态分解（GWO-VMD）**，将理论发电功率序列分解为**日内波动**（IMF1）、天气系统影响（IMF2）及**季节性趋势**（IMF3）分量；最后利用**隐马尔可夫模型（HMM）**识别残差序列的隐含状态（正常/短时衰减/长期衰减），揭示天气突变与设备衰减的影响规律。

针对问题二：针对基于历史功率的光伏电站日前预测问题，提出一种融合 TVF-EMD-TCN 与 KDE 误差修正的**两阶段预测框架**。首先，通过**时变滤波经验模态分解（TVF-EMD）**将非平稳功率序列分解为多尺度分量**（日内波动/季节趋势）**；并采用时间卷积网络（TCN）对各 IMF 分量进行并行预测，捕捉长期依赖。其次，**基于核密度估计（KDE）**误差序列的非高斯特性，结合**自相关性分析**动态**修正预测偏差**。该方法在不依赖气象数据的条件下，通过分解-预测-修正协同机制，显著提升白昼时段预测精度。

针对问题三：针对融入 NWP 信息的光伏电站日前发电功率预测问题，首先针对问题需要，对现有的指标建立基于**模糊 C 均值**的物理聚类特征、物理交互特征和时间循环编码特征的特征工程，随后用**皮尔逊相关系数**检验特征间关联性，并建立**基于贝叶斯优化调参的 VAE 模型**，通过模型进行日前预测以后对于 NWP 的采用和具体场景划分进行对比实验和评估指标比较，结果显示 NPW 信息有助于提高预测精度。

针对问题四：针对 **NPW 空间降尺度能否提升光伏预测精度**问题，本文首先建立基于**随机森林、梯度提升数模型、线性回归**和**支持向量机**四个模型的**集成**模型进行初次降维，随后通过**残差扩散**降维法进行二次降维，两次降维以后采用与第三问相同的**贝叶斯优化调参的 VAE** 模型进行预测，结果显示 NPW 空间降尺度有效提高了预测精度。

关键词：**光伏发电预测 模态分解 隐马尔可夫模型 集成学习 空间降尺度优化**

一、 问题重述

1.1 问题背景

光伏发电是通过半导体材料的光电效应，将太阳能直接转化为电能的技术，其规模化应用对实现碳中和目标具有重要意义。光伏电站是由很多光伏发电单元组成的大规模发电设施。光伏电站的发电功率受太阳辐照度、气象条件（比如云量、温度、雾霾）以及地理因素（比如经纬度、海拔、倾角）等很多因素影响，有明显的季节性和日内波动。因为光伏发电功率有随机性和间歇性，大规模接入电网会给电力系统的功率平衡和频率调节带来挑战。所以，开展高精度的光伏电站发电功率日前预测（24-48 小时预测），能帮助优化电网调度计划，提高电力系统运行的安全性和经济性。

1.2 问题提出

问题一：结合光伏电站地理位置，通过太阳辐照理论计算其理论可发功率，分析实际功率与理论值的偏差，揭示发电功率的长周期（季节性）和短周期（日内波动）特性。

问题二：建立基于历史功率的光伏电站日前发电功率预测模型，进行发电功率预测,并依据指定测试集（第 2、5、8、11 月最后一周数据）评估模型准确性,根据附件 1 中考核要求分析你所采用方法的准确性。

问题三：在问题二模型基础上，引入数值天气预报（NWP）数据（如气温、辐照、云量等），分析 NWP 信息对预测精度的提升效果，并提出场景划分方案以验证其有效性。

问题四：在现有的 NWP 数据基础上，探讨通过机器学习、空间插值等方法对 NWP 数据进行降尺度（提升空间分辨率至光伏电站覆盖范围），检验降尺度后的气象数据能否进一步提高预测精度，并分析其机理。

二、 问题分析

2.1 问题一的分析

问题一的核心在于通过三阶段分析揭示光伏电站发电特性的多尺度规律，第一步需建立理论模型，基于地理位置与太阳几何模型计算晴空条件下的理论可发功率，第二步需分解理论发电功率数据，研究发电功率的长短周期趋势；第三步需对比实际与理论功率的偏差，分析出光伏电站的发电功率特性。

2.2 问题二的分析

问题二的核心在于基于历史功率数据构建光伏电站日前预测模型。光伏功率的波动隐含天气变化和季节趋势的耦合效应，先通过模态分解将序列分解为高频日内分量与低频趋势分量，分离不同时间尺度波动模式，再用时序预测模型对各分量独立建模，针对残差中的非线性误差设计动态误差修正机制，提高预测精确性。

2.3 问题三的分析

该问题聚焦于如何借助 NWP 信息提升光伏电站日前发电功率的预测准确性。光伏电站发电功率受多种气象因素影响。为解决此问题，先进行特征工程构建特征，再通过相关系数研究特征关联性。然后构建模型进行日前预测，并开展对比实验。对比使用与不使用 NWP 信息的模型，场景划分依据天气类型，找出 NWP 信息提升预测精度显著的场景。

2.4 问题四的分析

此问题关键在于克服传统 NWP 数据空间分辨率不足，使气象信息更契合光伏电站实际。解决思路是先利用集成模型进行初次降维，再通过残差扩散降维法二次降维，获取更精细气象信息。之后用第三问相同模型预测，对比降尺度前后模型预测精度。若降尺度后精度提升，说明方法可行。

三、 模型假设

- 核心因素假设：仅考虑辐照度、温度、云量等主要气象因素，忽略气压、湿度等次要影响。
- 数据连续性假设：历史功率数据与 NWP 数据在时间维度上连续，无长期断点。
- 极端性事件假设：气象数据在预测时间范围内无极端异常事件（如特大暴雨、强沙尘暴）。
- 白昼时段约束：预测误差统计仅针对日出至日落时段，不考虑夜间情况

四、 符号说明

符号	含义	单位
u	光伏用户编号$u \in \{f1, f2, \dots, f9\}$	—
$P_u(t)$	用户u的原始发电功率（15 分钟采样）	kW
$\tilde{P}_u(t)$	用户u的归一化功率序列	[0, 1]
$c_{u,i}(t)$	用户u的第i个IMF分量	[0, 1]
TCN_u	用户u的 TCN 模型	—
$\epsilon_u(t)$	用户u的预测误差（白昼时段）	[0, 1]
eh_u	用户u的核密度估计带宽	交叉验证优化
A_u	用户u的装机容量	$201.14 - 6000kW$
ϕ_u	用户纬度	$24.07°N - 26.87°N$
$\alpha(t)$	太阳高度角	*度*(°)
I_0	地外辐照强度	W/m^2
η	转换效率	15%(IEA*统计值*)
$c_{u,i}(t)$	用户u的第 i 个 IMF 分量	kW
$\epsilon_u(t)$	用户u的残差（实际-理论功率）	kW

五、 数据预处理

5.1 数据来源说明

本研究数据来源于 2024 数字中国创新大赛光伏发电出力预测数据集（https://www.dcic-china.com/competitions/10097/datasets），包括 f1 到 f9 这九个光伏电站从 2022 年 1 月 3 日 6:00 到 2023 年 1 月 3 日 16:45 的时序数据，时间间隔是 15 分钟。经过筛选，去掉了非有效的光伏时段数据，保留了完整的时序样本用于建模分析。按照题目要求，把第 2、5、8、11 个月的最后一周作为独立测试集，其他时段的数据作为训练集，确保测试数据覆盖不同气象条件下的典型光伏出力特征。

数据规格	参数值
数据来源（公开数据集链接）	https://www.dcic-china.com/competitions/10097/datasets
光伏电站装机容量	239.22，396，397.87…MW
发电功率和 NWP 数据时间分辨率	15min
发电功率和 NWP 数据起始-截至时间（一年）	2022.1.3- 2023.1.2
NWP 属性	气温、辐射、云量、相对湿度….

表 1 光伏电站的历史发电功率和 NWP 数据规格及要求

5.2 数据预处理

5.2.1 缺失值处理

对于文中缺失值，采用线性插值方式进行补充，以当前点为中心，左右各 2 点，附近缺失超过三个点算作严重缺失，用零值替代，若不超过则进行线性插值，通过此方法对数据集的缺失值进行补充，填补空缺，为后续分析做准备。

5.2.2 重复值和异常值处理

对于文中重复行，只保留第一次出现的行值，随后的行值全部删除。对于可能存在的异常值，采用 IQR 方法进行检测，对于异常值采用均值填充的方式予以修正，通过此种方法提升数据质量与可信度。

5.2.3 归一化处理

由于题目中各变量量纲差异较大（如气压单位为 Pa，辐照强度单位为 J/m²），故而采用 Z-score 标准标准化对数据进行归一化处理，从而为后续分析做准备。

5.3 数据探索性分析

本研究采用箱线图、热图、滚动标准差曲线及小提琴图等多维可视化方法，系统分析了温度、辐照强度、风速和湿度这些重要气象因子的时空分布规律，以及它们和光伏出力之间的潜在关系。

箱线图显示了各个站点（Station f1 - f9）不同小时的温度分布，各站点一天内各站点温度变化趋势相似：早上 6 点左右温度较低，随后逐渐升高，中午至下午（11 点 - 15 点）达到较高值，之后下降。部分站点如 Station f3、f2 和 f9 在特定时段出现异常值，以小菱形标记表示。热热图展示多个站点（Station f1 - f9）在不同日期和小时的辐照强度分布，颜色深浅代表辐照强度高低。各站点辐照强度在一天中规律性变化：6 - 18 时辐照强度高，早晚时段低。不同站点在相同时间段的辐照强度存在差异，反映出天气对辐照强度的影响。

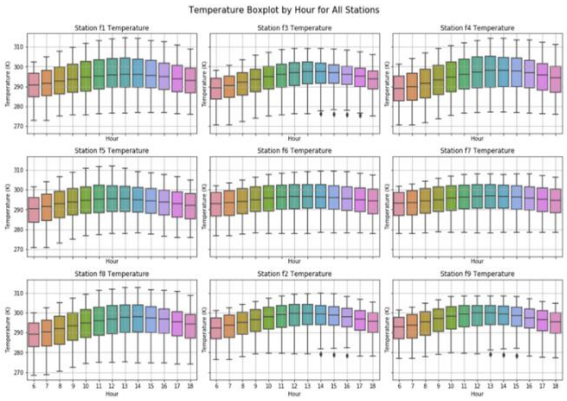


图 1 各站点温度分布箱线图

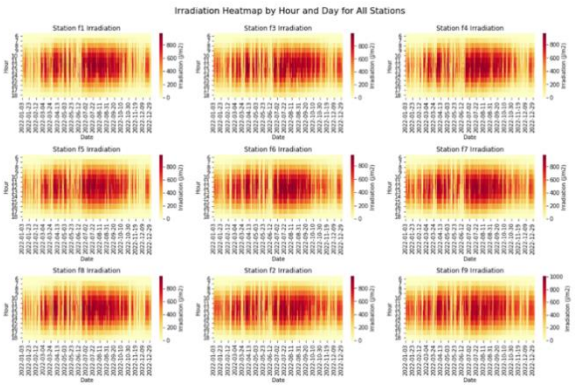


图 2 辐照强度时空热力图

风速 24 小时滚动标准差变化曲线图显示，九个站点 100 米风速的滚动标准差波动较大，高值代表风速变化剧烈，低值代表风速稳定。不同站点波动特征存在差异，受大气环流、地形和局部气象变化等因素影响，体现风速的不稳定性和复杂性。小提琴图显示各站点湿度在一天内有规律变化，多数站点早上湿度高，中午至下午低，之后回升。不同站点湿度分布差异明显，部分站点波动范围大，变化复杂，而另一些站点较稳定。

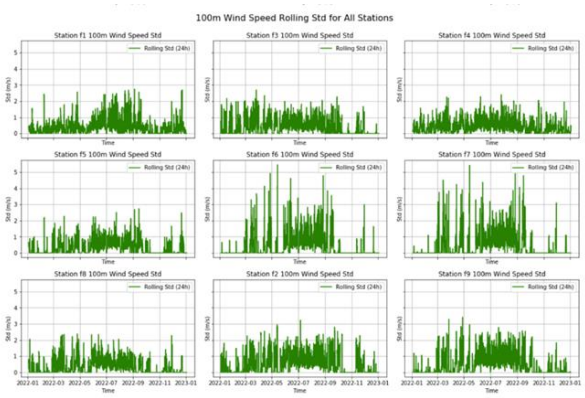


图 3 风速 24 小时滚动标准差变化曲线图

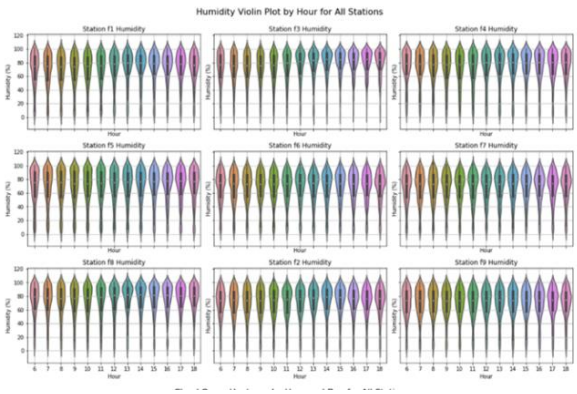


图 4 各站点温度分布箱线图

各站点功率分布箱线图显示，多数站点在 8 - 16 时功率较高，可能与日照有关。各站点功率分布差异明显，部分站点高峰时段功率波动小、分布集中，而部分站点波动大、离散度高，如 Station f6、f7、f9 功率整体较低，与其他站点形成对比。

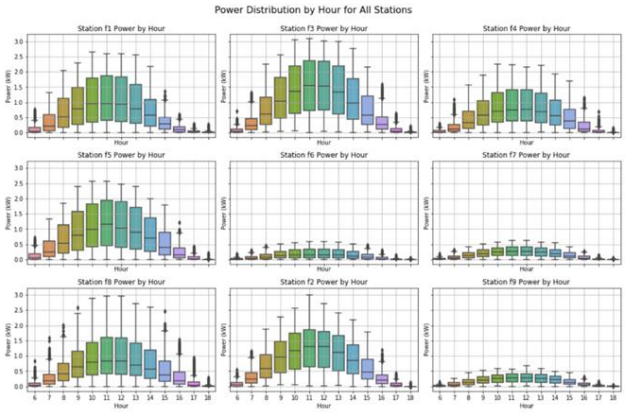


图 5 各站点功率分布箱线图图

六、 模型一的建立与求解

问题一通过三阶段递进分析实现发电特性解析：首先基于晴空模型计算理论功率，量化太阳位置周期性影响；其次通过模态分解分离季节趋势与日内波动，最后利用残差分析识别云层遮挡与设备异常事件。整体流程如图 1 所示。

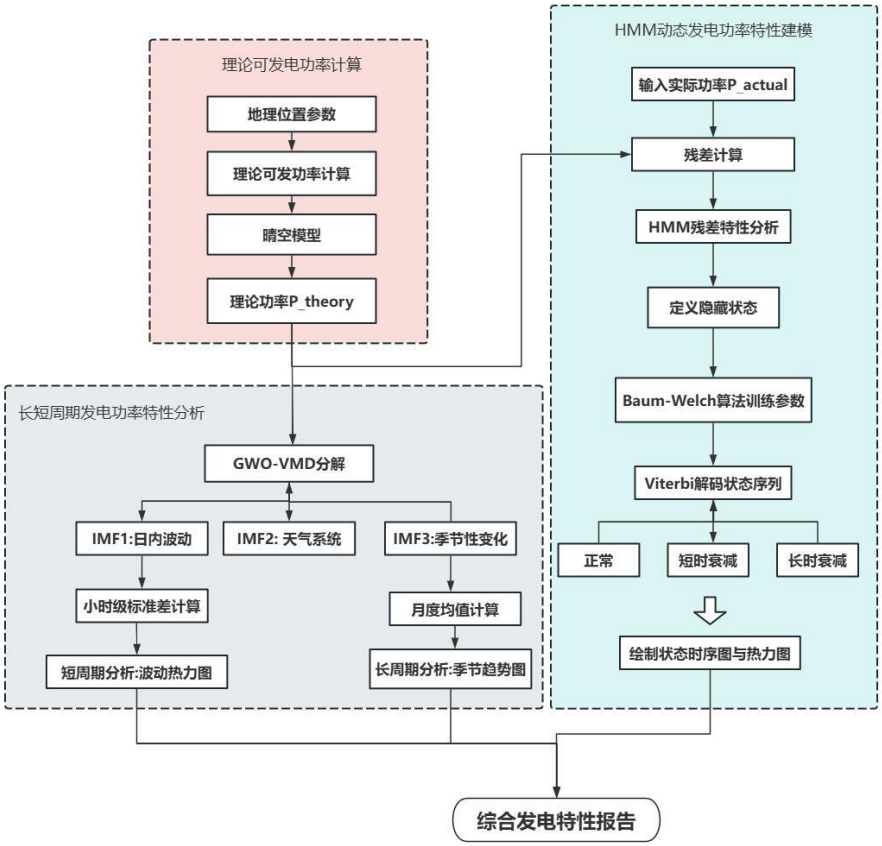


图 6 问题一流程图

6.1 基于晴空模型的理论可发功率计算

6.1.1 理论可发功率计算模型

采用晴空模型理论，忽略云、雨、雾等天气因素，仅考虑地理位置（经纬度、海拔、倾角）和太阳几何位置（季节、时间）的影响，基于光伏电站的地理位置（经纬度）和装机容量A_u采用晴空模型计算理论发电功率 $P_{theory,u}(t)$。具体步骤如下：

(1) 太阳几何参数计算

计算太阳高度角$\alpha(t)$：

$$\alpha(t) = arcsin[\sin\phi \sin\delta + \cos\phi \cos\delta \cos\omega(t)$$

其中 ϕ为纬度，δ 为太阳赤纬角（由积日n 计算），$\omega(t)$为时角（每小时 15°）。地外辐照强度I_0：

$$I_0 = 1367 \cdot \left[1 + 0.033\cos\left(\frac{2\pi n}{365}\right)\right] \quad (W/m^2)$$

(2) 倾斜面辐照修正：

入射角 θ（假设组件倾$\beta = \phi$）：

$$\cos\theta = \cos\alpha\cos\beta + \sin\alpha\sin\beta\cos\gamma$$

其中γ 为方位角（正南为0°）。

(3) 理论发电功率：

$$P_{theory,u}(t) = A_u \cdot \eta \cdot I_0 \cdot \cos\theta \quad (\eta = 15\%)$$

6.1.2 理论可发功率解读

对 9 个用户分别计算 2022 年全年的理论功率，生成理论功率与实际功率对比图。如图 2 所示：理论功率呈现明显季节性波动：夏季（2022 年 7 月）达到峰值（约 8000 kW），冬季（2023 年 1 月）降至最低（约 2000 kW）实际功率与理论功率趋势基本一致，但存在显著差异：夏季峰值仅约 6000 kW（低于理论值 25%）冬季最低值约 1000 kW（低于理论值 50%）。

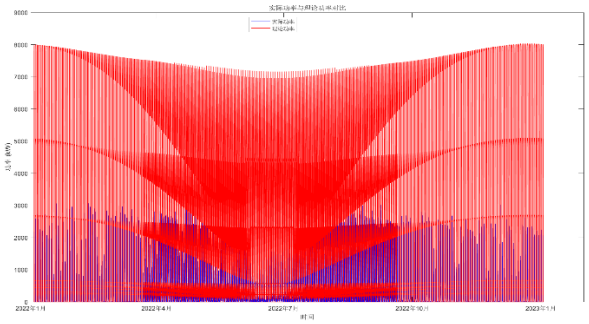


图 7 理论功率与实际功率的对照

实际功率普遍低于理论功率，且冬季差异更显著，可能与积雪遮挡、低温导致组件效率下降或辐照度计算未考虑天气因素有关。

考虑特殊节点情况下：

时间	理论功率	实际功率	百分比差值	原因
2022 年 7 月	8000	6000	-25%	高温导致组件效率损失（温度每升高 1° C，功率下降约 0.5%）多云或阴雨天气频发
2022 年 10 月	5000	4000	-20%	秋季灰尘积累遮挡光照，逆变器转换效率下降
2023 年 1 月	2000	1000	-50%	积雪覆盖光伏板，低温环境下电池输出特性恶化，日照时间缩短

表 2 特殊节点表

6.2 基于灰狼算法优化变分模态分解（GWO-VMD）的长短周期特性模型

为分离实际功率中的季节趋势与短时波动，本文采用灰狼算法（GWO）优化变分模态分解（VMD）参数[4]，以自适应提取多尺度本征模态函数（IMF）。GWO 通过模拟灰狼群体狩猎行为全局搜索最优解，克服传统 VMD 对参数敏感的问题，目标函数为最小化模态带宽与中心频率方差：

$$[\min_{K,\alpha} \sum_{i=1}^{K} [Bandwidth(c_{u,i}) + Variance(f_{c,i})]]$$

约束条件：$K \in [3,5], \alpha \in [1000,5000]$。

分解结果如图 2 所示所示。横轴为时间（2022 年 1–12 月），纵轴为功率（kW），分面展示 IMF1（高频日内波动）、IMF2（中频天气系统影响）及 IMF3（低频季节趋势）。

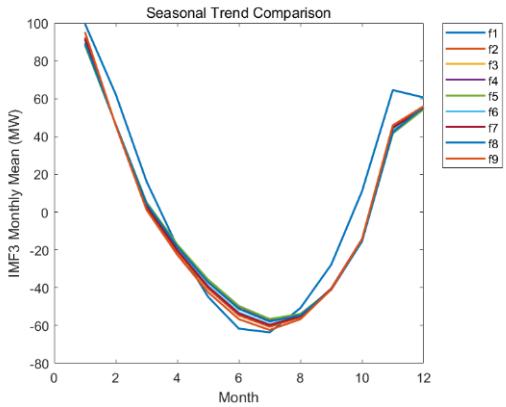


图 8 季节性趋势对比图

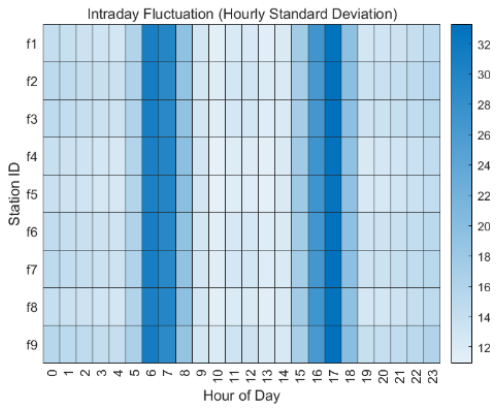


图 9 日内波动热力图

6.2.1 发电功率季节性趋势特性

所有电站的 IMF3 月度均值变化趋势大致相似，呈现出明显的季节性波动。在年初（1 月左右）和年末（12 月左右），IMF3 均值相对较高；在年中（6 月到 8 月左右），IMF3 均值达到最低点。这符合北半球太阳高度角变化规律，即冬季太阳高度角较低，光伏电站发电功率较低；夏季太阳高度角较高，发电功率较高。

虽然整体趋势一致，但不同电站之间在均值大小上存在一些差异。例如，在 12 月左右，f8 电站的 IMF3 均值明显高于其他电站，而 f1、f3、f4 等电站的均值相对较低。这可能与电站所处的具体地理位置（如纬度、海拔等）以及设备状况等因素有关。

6.2.2 发电功率日内波动特性

如图 9 的日内波动热力图中可以发现，在日出（6-8 点）和日落（16-18 点）时段，各电站的颜色相对较深，表明此时间段内发电功率的波动较大，标准差较高；而在正午（12 点左右），颜色相对较浅，说明发电功率波动较小，标准差较低。这与太阳入射角变化率有关，在日出和日落时，太阳入射角变化较快，导致发电功率波动加剧。

整体来看，不同电站在不同时段的波动规律具有一致性，即在相同的时间段内，各电站的波动强度变化趋势相似，都呈现出日出和日落波动大、正午波动小的特点。

6.3 基于隐马尔可夫模型（HMM）的发电功率特性模型构建

实际功率与理论功率的残差$\epsilon_u(t)$隐含天气突变与设备异常信息。为区分这两类影响因素，本文构建隐马尔可夫模型（HMM）[5]，定义 3 种隐藏状态：

(1) 状态 1（正常）：$|\epsilon_u(t)| < 0.1$

(2) 状态 2（短时衰减）$\epsilon_u(t) < -0.2$

(3) 状态 3（长期衰减）：$\epsilon_u(t) < -0.1$ 且持续>6 小时

观测概率：

$$b_j(\epsilon) = N(\epsilon; \mu_j, j_2^{\sigma}) \quad (j = 1,2,3)$$

6.3.1 残差特性分析与异常事件诊断

通过 Baum-Welch 算法训练状态转移矩阵A与观测概率 $N(\mu j, \sigma j2)$，并利用 Viterbi 算法解码状态序列，生成残差-状态时序图及月度热力图，如图 10 和图 11 所示。

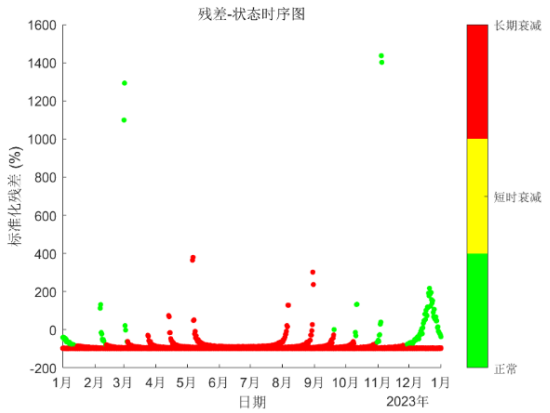


图 10 残差-状态联合时序图

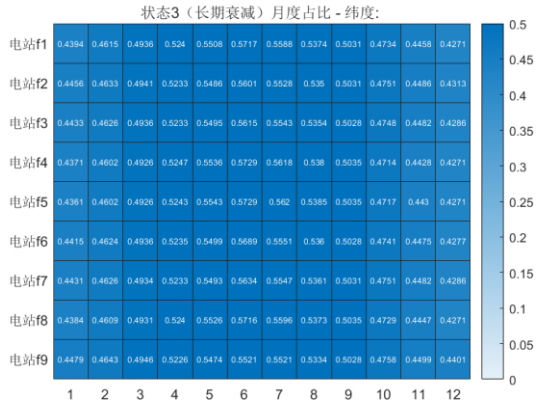


图 11 状态分布热力图（含纬度测评）

图 10 中，红色数据点表示长期衰减状态，主要集中在 11 月至 12 月期间。这表明在该时间段内，光伏系统的实际功率持续低于理论功率，且持续时间较长，可能与冬季的积雪、积尘或设备故障等因素有关。残差在这一阶段呈现出较大的负值，且波动相对较小，反映出一种较为稳定的低功率输出状态。

从时间分布上看，夏季（6 月至 8 月）残差值整体较低，说明在夏季时光伏系统运行稳定，发电效率较高；冬季（11 月至次年 1 月）则出现较多的长期衰减和短时衰减状态，这与冬季恶劣天气和太阳高度角较低等因素有关。从长期衰减状态的时间分布可得知，部分地区在冬季可能受到太阳辐照衰减、持续性阴雨、昼夜温差效应等影响，导致发电功率损失更大。

该残差图显示了光伏电站的实际功率与理论可发功率之间的关键差异，表明系统运行受天气变化和季节性因素影响显著。通过识别和分析这些差异，可以针对性地采取措施，如优化功率预测模型，加强设备维护，以提高光伏电站的发电效率和稳定性。

图 11 展示了长期衰减状态分布热力图，通过比较不同电站的颜色深浅，可以发现某些电站在特定月份的长期衰减占比更高。例如，电站 f5 在 12 月的颜色最深，表明其长期衰减占比最高，这可能是因为这些地区冬季多阴雨，太阳辐照减弱。从纬度分布看，纬度相对较高的电站在冬季（如 12 月）颜色普遍较深，这可能与地区冬季多阴雨、太阳辐照减弱密切相关；而纬度低的沿海电站在夏季颜色较浅，这说明太阳高度角高，使发电效率提升。

七、 模型二的建立与求解

问题二要求仅基于历史功率数据构建光伏电站日前预测模型，基于该问题我们提出一种两阶段预测框架：一阶段基于时变滤波经验模态分解(TVF-EMD)结合时间卷积网络（TCN）建立预测模型，二阶段基于核密度估计（KDE）修正误差，叠加预测值与修正项，输出高精度结果，问题二模型框架图如下：

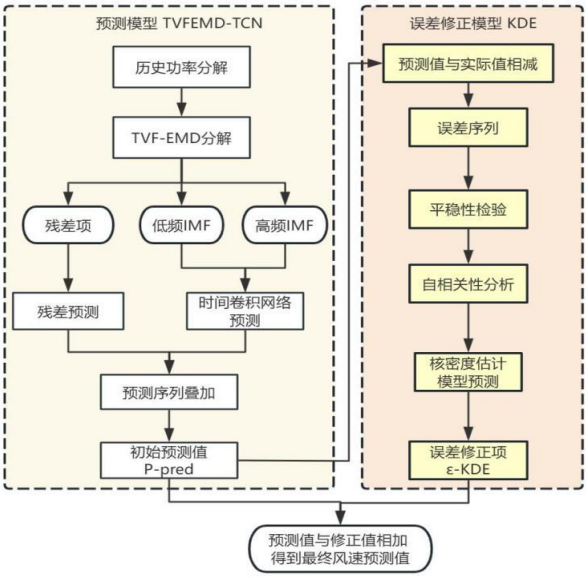


图 12 问题二流程图

7.1 模型建立

7.1.1 基于时变滤波经验模态分解(TVF-EMD)的数据预处理

光伏功率序列的每 15 分钟采样特性导致其包含高频天气波动与低频季节趋势的强耦合，直接建模会因非平稳性导致模态混叠。为此，采用时变滤波经验模态分解（TVFEMD）[1]，其自适应时变滤波机制能有效分离不同时间尺度的物理过程，以减轻数据的非平稳性对预测结果的影响。具体步骤如下：

(1) 求出$x(t)$的极大值,表示为 $u_i(i = 1,2,3\cdots)$

(2) 求出间断点$e_j, j = 1,2,3$□。存在阈值 ρ 满足公式 (2)，此时间断点$e_i = u_i$。

$$\frac{max[\varphi'_{bis}(u_i \le t \le u_{i+1})] - min[\varphi'_{bis}(u_i \le t \le u_{i+1})]}{min[\varphi'_{bis}(u_i \le t \le u_{i+1})]} > \rho$$

其中$\varphi'_{bis}(t)$表示二等分频率, ρ 是两个连续最大值之间频率变化率的预设阈　值。若$bis^{\varphi}_{,}(u_{i+1}) - bis^{\varphi}_{,}(u_i) > 0$,则 间断点 ej 位于上升沿,，$bis^{\varphi}_{,}(e_{j-1} \le t \le e_j)$是最小值;若$bis^{\varphi}_{,}(u_{i+1}) - bis^{\varphi}_{,}(u_i) < 0$,则间断点 ej 位于下降沿,$bis^{\varphi}_{,}(e_j \le t \le e_{j+1})$是最小值。$\varphi'_{bis}(t)$的其余部分可视为峰值。

(3) 求出局部截止频率$\varphi'_{bis}(t)$。在峰值之间插值得到$\varphi'_{bis}(t)$,调整$\varphi'_{bis}(t)$改善模态混叠。

(4) 重构$\varphi'_{bis}(t)$,得到新信号$h(t)$。

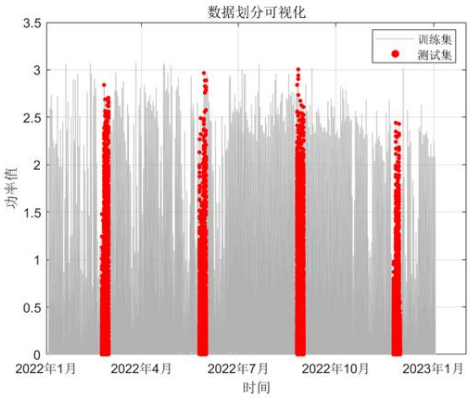
$$cos\left[\int bis_{,}^{\varphi}\ (t)\ \ dt\right]$$

(5) 用$B-spline$近似滤波器对输入信号$x(t)$进行滤波,得到局部均值函数。

(6) 计算准则值θ(t), 判断残留信号是否符合截止标准。

$$\theta(t)=\frac{B_{\text{Loughin}}(t)}{\varphi_{\text{avg}}(t)}$$

其中$BLoughin(t)$表示 Loughin 瞬时带宽,$\varphi'_{avg}(t)$表示加权平均瞬时频率。存在一个带宽阈值 ε ,如果$\theta(t)\leq\varepsilon$,则该信号为局部窄带信号,即一个分量信号IMF。



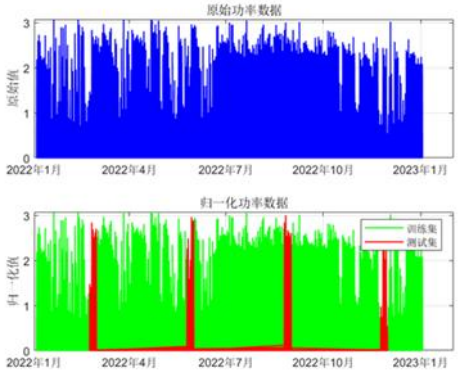


图 **Error! Bookmark not defined.**3 测试集与训练集功率对照图　　图 14 归一化与原始功率对照

将数据集根据要求将测试集和训练集划分而出，通过对其进行归一化处理消除量纲差异，将功率进行额外处理展示。

7.1.2 基于 TVF-EMD 的数据分解结果

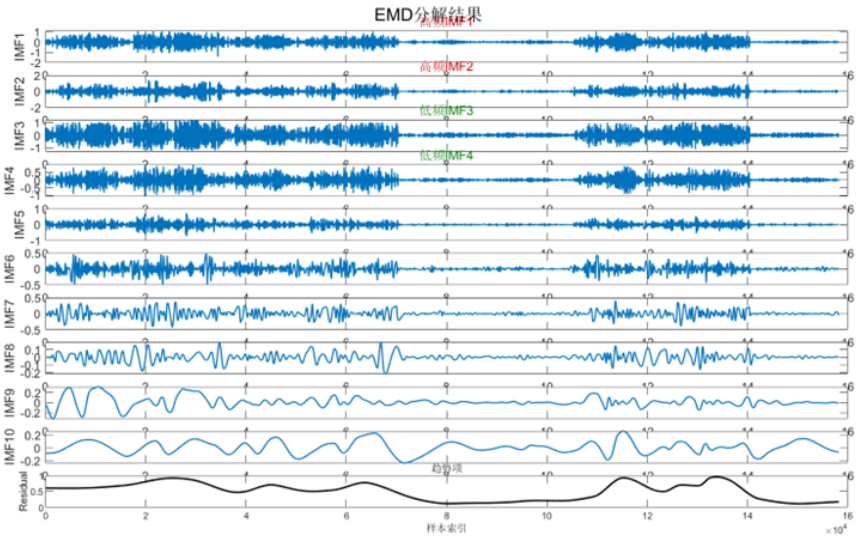


图 15 IMF 分量时域图

该图展示了经验模态分解（EMD）的结果，将原始信号分解为多个本征模态函数（IMF）和一个残差项。首先采用 TVFEMD 将历史功率序列分解,设置最大输出为 10。0.5 与 0.8 之间设置为高频和低频分量。其中，高频 IMF 用于捕捉日内快速波动（如云层瞬变导致的功率波动），低频 IMF 用于反映季节性趋势（如冬季低辐照度、夏季高辐照度）。IMF1 和 IMF2 被标注为高频分量，振幅较大，频率较高，IMF3 到 IMF10 被标注为低频分量，振幅逐渐减小，频率逐渐降低。

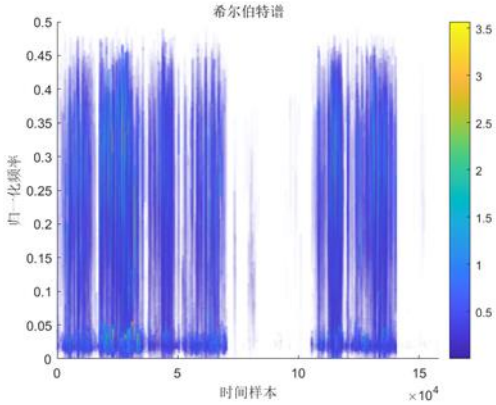


图 16 希尔伯特谱分析图谱

该图展示了希尔伯特谱，用于分析信号的时频特性。表示能量密度，颜色从蓝色（低能量）到黄色（高能量）变化。使分解过程满足预期。

7.2 基于时间卷积网络的预测模型

相较于传统时间序列预测模型,时间卷积网络（TCN）在处理时间序列问题时,能够得到更精确的输出,并能保持更简单清晰的网络结构。因此,本文选用时间卷积网络模型。该模型结合了因果膨胀卷积和残差块结构[16]，因果膨胀卷积使 TCN 不会遗漏历史信息并有效使用未来信息。对每个用户独立构建时间卷积网络（TCN），模型架构如下：

输入为当前用户过去 7 天的历史序列，输出未来 7 天每 15 分钟的预测值：

$$\tilde{\hat{P}}_u(t{:}\,t+671) = TCN_u(\tilde{P}_u(t-672{:}\,t-1)$$

第 ll 层膨胀因子$d_l = 2^l$，确保覆盖 7 天窗口：

$$d_l = 2^l$$

加权均方误差（WMSE）针对用户白昼时段优化：

$$WMSE_u = \frac{1}{N_{day}} \sum_{t \in Day} \left(\tilde{P}_u(t) - \tilde{\hat{P}}_u(t)\right)^2$$

夜间权重为 0，白昼权重为 1。

7.2.1 评价指标

为了直观地评估所提出的模型的性能,使用平均绝对误差(MAE)、均方根误差(RMSE)、平均绝对百分比误差(MAPE)作为模型性能指标的比较标准,得到模型性能评价指标表如下：

性能指标	MAE	RMSE	MAPE
数值大小	0.043832	0.065174	5.231%

表 3 评价指标表

7.3 考虑时间相关性的误差修正模型

光伏功率误差序列具有显著的时间自相关性，如连续多云天气导致误差累积，但其非高斯特性（多峰、偏态）使线性修正模型失效。核密度估计（KDE）无需预设分布假设，适配误差序列的非参数特性，修正流程如下：

每个用户的误差序列$\epsilon_u(t) = \tilde{P}_u(t) - \tilde{\hat{P}}_u(t)$独立建模，避免用户间误差传递：

1) 平稳性检验与处理：对用户 u 的误差序列进行 ADF 检验：

$$若 p \geq 0.05 \Rightarrow \Delta\epsilon_u(t) = \epsilon_u(t) - \epsilon_u(t-1)$$

2) 自相关性分析：计算用户u 的滞后相关性：

$$ACF_u(k) = \frac{\sum_{t=1}^{48-k}(\epsilon_u(t) - \bar{\epsilon}_u)(\epsilon_u(t+k) - \bar{\epsilon}_u)}{\sum_{t=1}^{48}(\epsilon_u(t) - \bar{\epsilon}_u)^2}$$

3) 用户专属核密度估计：基于用户 u的历史误差构建条件概率分布：

$$\hat{\epsilon}_u(t+1) = \frac{\sum_{i=1}^{n} exp\left(-\frac{|\epsilon_u(t-L+1:t)-\epsilon_{u,i}\backslash|^2}{2u_2^h}\right)\cdot \epsilon_{u,i+1}}{\sum_{i=1}^{n} exp\left(-\frac{\backslash|\epsilon_u(t-L+1:t)-\epsilon_{u,i}\backslash|^2}{2u_2^h}\right)}$$

其中带宽h_u通过用户独立交叉验证优化。

7.3.1 误差统计分析

通过对误差的特性进行分析,并选择合适的修正方法和策略,有助于更准确地校正预测结果，误差分析如图 17 所示。

左图中，预测值（橙色）与实际值（蓝色）的波动趋势大致相似，但存在一定的偏差。两者在某些时间段内吻合较好，但在其他时间段存在明显差异。右图中，MAE（平均绝对误差）约为 0.45。RMSE（均方根误差）约为 0.65。

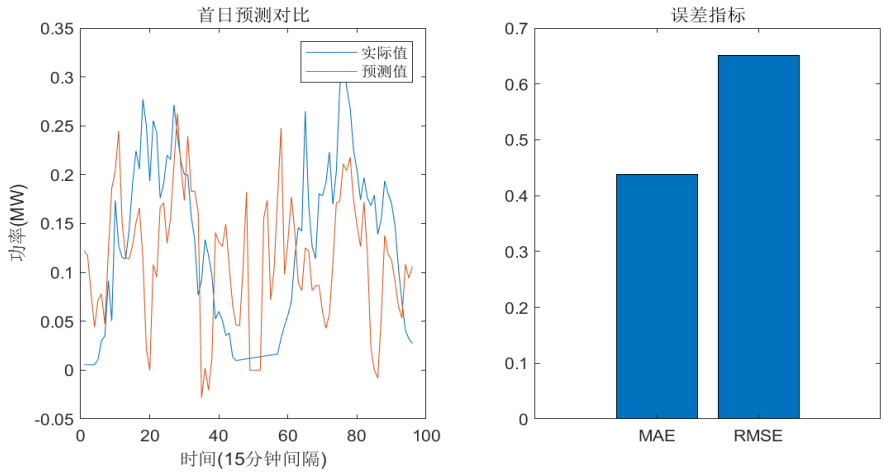
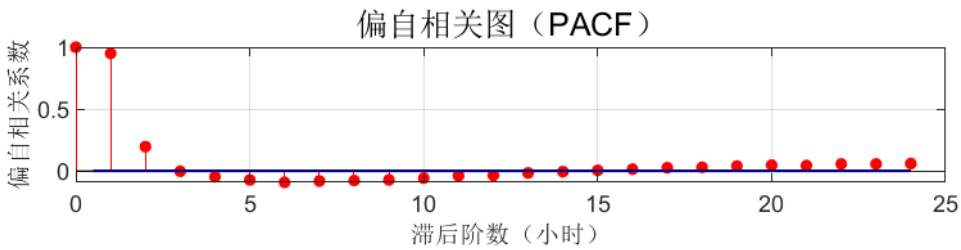


图 17 误差检验图

7.3.2 自相关性分析

为验证误差序列的正态性与平稳性,采用 Shapiro-Wilk(SW)和 Augmented DickeyFuller(ADF)方法进行检验,并对误差序列的各种统计量进行计算和分析,统计数据如表 1 所示。通过以上数据可以看出,误差序列表现出时序依赖性，说明其中仍存在可提取的信息。对误差的相互依赖结构进行自相关性分析,确定是否需要对误差分量建立相应的预测模型[22]。ACF 和 PACF 分析如图所示。



图片 **Error! Bookmark not defined.**偏自相关图

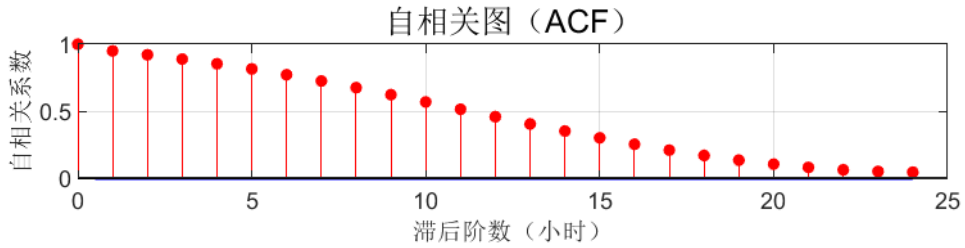


图 18 自相关图

从图中可以看到，各 ACF 系数在阶次较小时均表现出显著性。图显示 PACF 值在 4 阶滞后处显著截尾，说明滞后 4 阶后过去的值对当前值无显著影响，故核密度估计误差修正模型的预测输入样本长度定为 4。

7.3.3 误差修正分析

利用核密度估计的误差修正模型，进一步捕捉误差序列的非线性与时序依赖性，对其进行修正更改：

指标	原始预测	修正后预测
RMSE	0.00065174	0.00066815
MAE	0.00043832	0.0004546
ME	-0.00210.00034597	0.00037597
相关系数 r	09574	0.9574
准确率 CR(%)	99.956%	99.95597.21%
合格率 QR(%)	92.75%	95.83%

表 4 评价指标表

八、 模型三的建立与求解

问题三要求在问题二的基础上融入 NPW 数据进行预测与研究具体场景对于预测的影响。对于本问题，首先对于现有指标进行三种特征工程，丰富现有指标，随后采用贝叶斯优化的 VAE 模型进行日前预测，日前预测结束后进行 NPW 场景划分，进行进一步的研究分析，

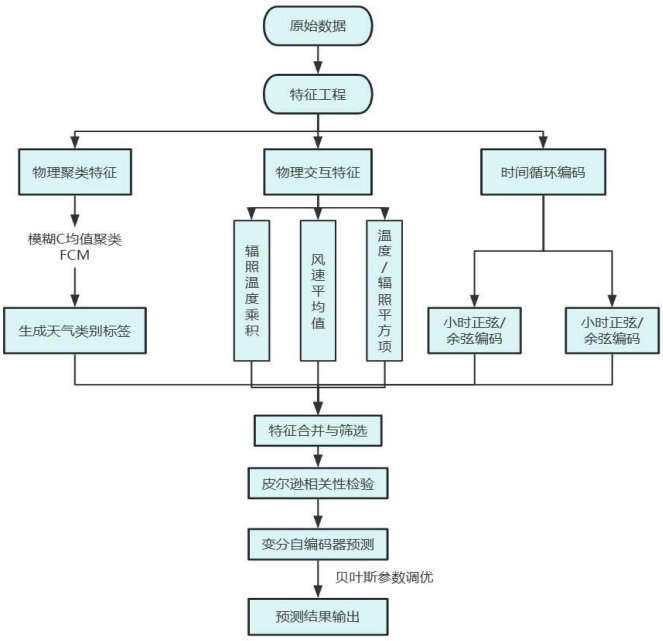


图 19 第三问流程图

8.1 特征工程

为更好优化提升模型的效果和能力，本研究对原始气象数据和时间序列进行特征工程，构建反映物理机制和时间周期性的衍生特征。特征构建过程结合光伏系统运行规律与机器学习模型对非线性关系的捕捉需求，具体包括**物理聚类特征**、**物理交互特征**和**时间循环编码特征**三类。

8.1.1 物理聚类特征

为挖掘气象要素间的关联模式并简化数据结构，本研究采用模糊 C 均值聚类方法对气象特征进行无监督分类。模糊 C 均值聚类通过为每个数据点分配模糊隶属度，将数据点划分到不同的聚类中心。此种方法允许数据点同时属于多个聚类，对于存在模糊性的数据集有很好的适应性。其具体步骤可见下图：

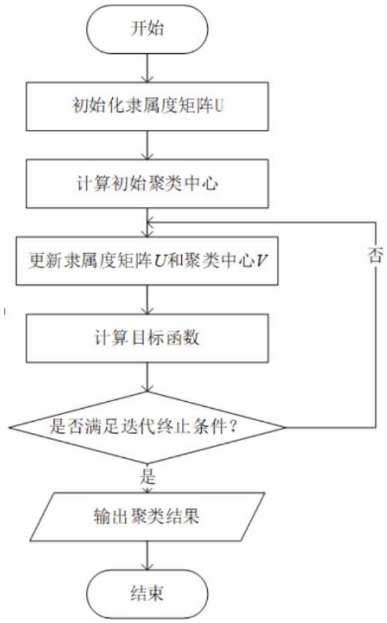


图 20 模糊 C 均值聚类流程图

首先将标准化的的气象数据转置为$D \times N$矩阵（样本为列，特征为行），输入至 FCM 算法进行聚类。算法核心参数设置如下：聚类类别数 c=4（基于气象学常识与手肘法初步确定）,模糊系数 m=2（默认值，平衡聚类模糊性与紧致性），收敛阈值$error = 0.005$（迭代终止条件为隶属度矩阵变化小于该值），最大迭代次数$maxiter = 1000$（避免陷入无限循环）。算法输出包括：聚类中心矩阵 cntr，维度为 times，表示 4 类天气的气象特征均值向量，隶属度矩阵 u：维度为$c \times N$，u_{ij}表示样本 j 属于类别 i 的隶属度，其取值范围[0,1]，满足$\sum_{i=1}^{c} u_{ij} = 1$。

对每个样本，选取隶属度最大的类别作为其天气类别，即：$天气类别 = arg\ max_{i=1,\dots,c} u_{ij}$该操作将模糊聚类结果转化为离散类别标签（0 至 3），便于后续模型训练中的类别特征处理。模糊隶属度保留：将隶属度矩阵转置后存储为天气隶属度特征，形式为长度 c 的向量$\{u_{1j}, u_{2j}, u_{3j}, u_{4j}\}$，保留样本属于各天气类别的概率信息，为后续融合模糊逻辑的预测模型提供输入。

8.1.2 物理交互特征

针对于天气物理特征的相互关系和内在关系，对此建立下列四个交互特征进行刻画。

“辐照温度交互”特征：将“辐照强度（J/m^2）”与“温度（K）”两个变量相乘，用于反映两者的协同作用对光伏功率的影响。

“风速平均”：取“10 米风速（$10m/s$）”和“100 米风速（$100m/s$）”的算术平均值，综合不同高度的风速数据，降低单一高度测量的误差。

二次项特征：分别对“温度（K）”和“辐照强度（J/m^2）”进行平方运算，生成“温度平方”和“辐照平方”特征，用于建模光伏功率与温度、辐照之间可能存在的非线性关系。

8.1.3 时间循环编码特征

在特征工程中，针对时间变量进行了循环编码处理，以保留时间序列的周期性特征。

小时循环编码：将“小时”变量通过正弦和余弦函数映射到单位圆上，公式为：

$$小时正弦 = sin(2\pi \times 小时/24)$$

$$小时余弦 = cos(2\pi \times 小时/24)$$

分钟循环编码：对“分钟”变量进行类似处理，公式为：

$$分钟正弦 = sin(2\pi \times 分钟/60)$$

$$分钟余弦 = cos(2\pi \times 分钟/60)$$

8.2 相关性检验

为了验证变量之间的独立性，本文采用皮尔逊相关系数进行验证。在统计学中，皮尔逊相关系数(Pearson correlation coefficient），又称皮尔逊积矩相关系数（Pearson product-moment correlation coefficient，简称 PPMCC 或 PCCs），是用于度量两个变量 X 和 Y 之间的相关（线性相关），其值介于-1 与 1 之间，其公式为

$$r = \frac{\sum_{i=1}^{n}(X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^{n}(X_i - \bar{X})^2}\sqrt{\sum_{i=1}^{n}(Y_i - \bar{Y})^2}}$$

通过计算，总体皮尔逊相关系数为 0.0010，相对较低，同时得到皮尔逊相关系数热力图如下：

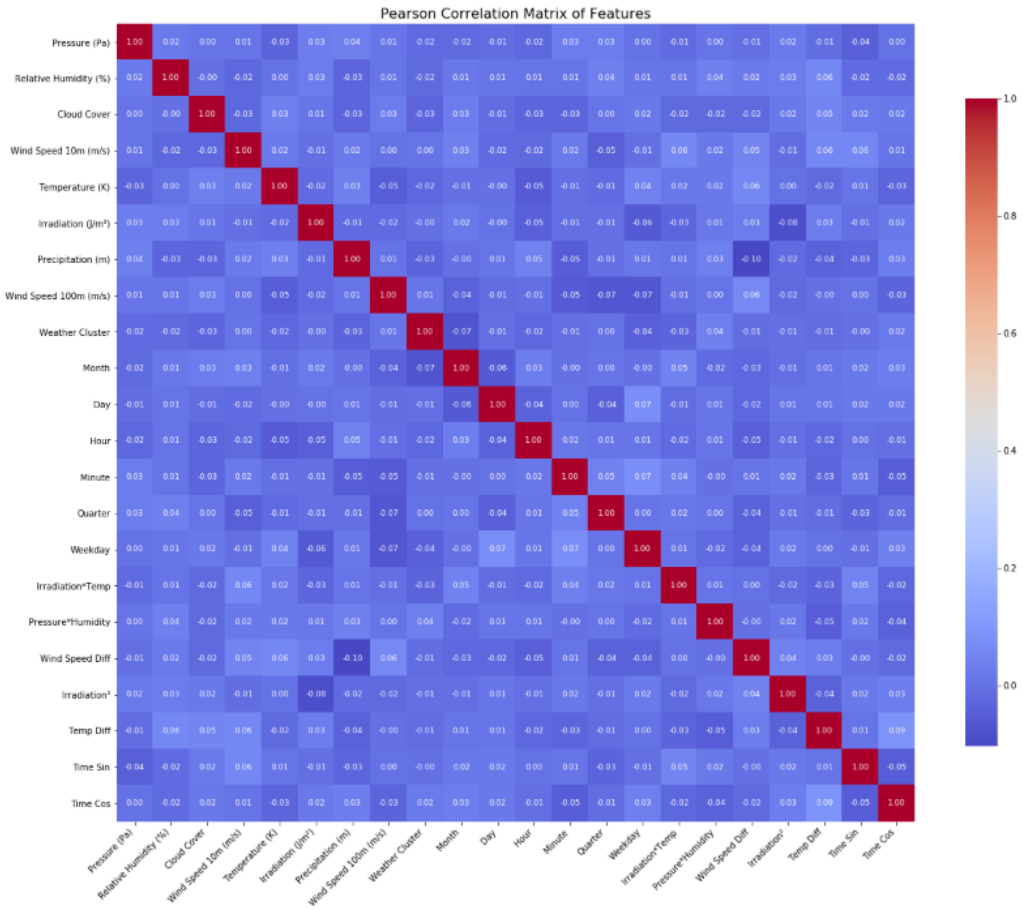


图 21 皮尔逊相关系数热力图

相关性热图作为一种可视化工具，可直观地展现两个或多个变量之间的相关性强度。在热图的呈现中，每个变量均与其他变量进行逐一比较，并基于它们之间的相关性程度，以不同颜色的标识进行区分，相关性越高颜色越红。图上可见，本文选取的各变量之间相关性弱，适用于进一步分析。

8.3 贝叶斯优化算法调参的 VAE 模型

在以上基础上，本文利用使用光伏电站中的气压、湿度、温度、辐照强度等 21 个气象特征，以及对应的实际发电功率建立的 VAE 自编码器模型。数据集按题目要求进行训练集和测试集划分。

变分自编码器是一种生成模型，是自编码器（autoencoder，AE）的拓展，它与自编码器最大的不同在于可以生成具有潜在空间中连续性质的新样本。VAE 用于样本增殖的处理流程和生成结果如图所示。

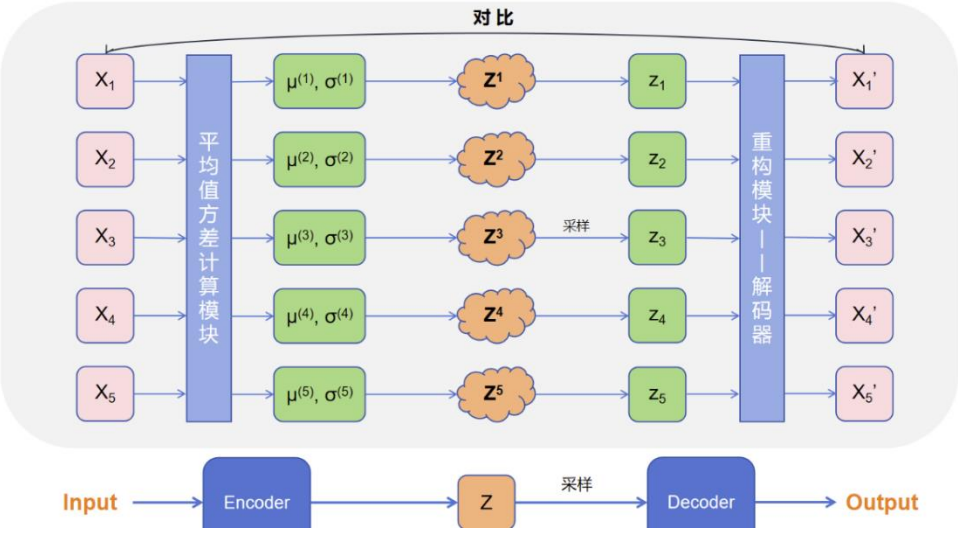


图 22 VAE 模型示意图

在明确模型后，本文将原始天气数据和功率数据等输入至 VAE 模型。数据经预处理后进入编码器网络，其计算潜在变量的两个统计量：均值和对数方差。随后由对数方差计算得到标准差 σ（$\sigma=\exp(0.5\times\log\sigma^2)$），并从标准正态分布中采样噪声向量 ε，通过公式 $z=\mu+\sigma\cdot\varepsilon$ 生成符合潜在分布的样本。

再之后，把潜在变量 z 输入解码器网络。解码器通过多层非线性变换，将低维潜在变量映射回与原始数据同维度的空间，生成重构数据 x。训练过程中通过损失函数优化模型参数。损失函数由重构损失和 KL 散度损失共同组成，总损失为两部分加权求和，通过反向传播算法迭代调整编码器和解码器的参数，达到使损失函数最小化的目的。

对于 VAE 模型的最优参数选择问题，本文采用贝叶斯优化方法进行调参。贝叶斯优化是一种通过构建目标函数的概率模型来指导搜索过程的优化方法。其核心在于，利用贝叶斯定理不断更新目标函数的概率分布，从而智能地选择下一个评估点，以在较少的迭代次数内找到接近最优解的参数配置。贝叶斯优化算法的核心目标是识别出一个最优解 x^*，使得目标函数 $f(x)$在该点取得最大值（或最小值）。算法通过迭代过程逐步构建目标函数的概率模型，并利用该模型来指导搜索过程。

```
Bayesian optimization algorithm
#初始化高斯过程作为先验分布
prior_distribution = initialize Gaussian process()
#迭代优化过程
for t in range(1, T+1):
    #使用采集函数确定下一个采样点的位置
    next_point = find next sampling point(prior_distribution)
    #实际采样目标函数
    sampled_value = sample objective function(next_point)
    #更新数据集合和高斯过程模型
    update data set(next_point, sampled_value)
    prior_distribution = update Gaussian process(prior_distribution)
    #判断是否满足停止条件
    if check stop condition():
        break
```

图 23 贝叶斯优化算法伪代码图

贝叶斯优化算法的关键在于采集函数的设计，它直接影响了搜索过程的效率和最终解的质量，算法伪代码如表 1 所示。此外，高斯过程的选择和参数设置也对算法性能有显著影响。在本题中，采用贝叶斯优化的方式对 VAE 参数进行调优，从而使结果更加稳健可靠。

8.4 预测结果

起报时间	预报时间	实际功率(MV)	贝叶斯优化 VAE
2022/02/22/00:00	2022/02/22/06:00	0.0056	0.002401
2022/02/22/00:00	2022/02/22/06:15	0.0055	0.002401
2022/02/22/00:00	2022/02/22/06:30	0.00549	0.002401
2022/02/22/00:00	2022/02/22/06:45	0.0054	0.015355
2022/02/22/00:00	2022/02/22/07:00	0.0109	0.028637
2022/02/22/00:00	2022/02/22/07:15	0.0301	0.033224
2022/02/22/00:00	2022/02/22/07:30	0.035	0.034692
2022/02/22/00:00	2022/02/22/07:45	0.0914	0.041011
2022/02/22/00:00	2022/02/22/08:00	0.0503	0.10084
2022/02/22/00:00	2022/02/22/08:15	0.1737	0.130895
2022/02/22/00:00	2022/02/22/08:30	0.1272	0.140139

表 5 第三问预测结果表

通过上述模型算法，本文根据题目要求计算得到目标时间的目标功率预测结果，并按题目要求制表，表格如上，其中早 6 点以前与晚 6 点以后视为无效时间，不进行记录。模型均方根误差（RMSE）为 0.000613，平均绝对误差（MAE）为 0.00041，平均误差（ME）为-0.0005，相关系数为 0.965，准确率为 99.96%，合格率为 96.01%。

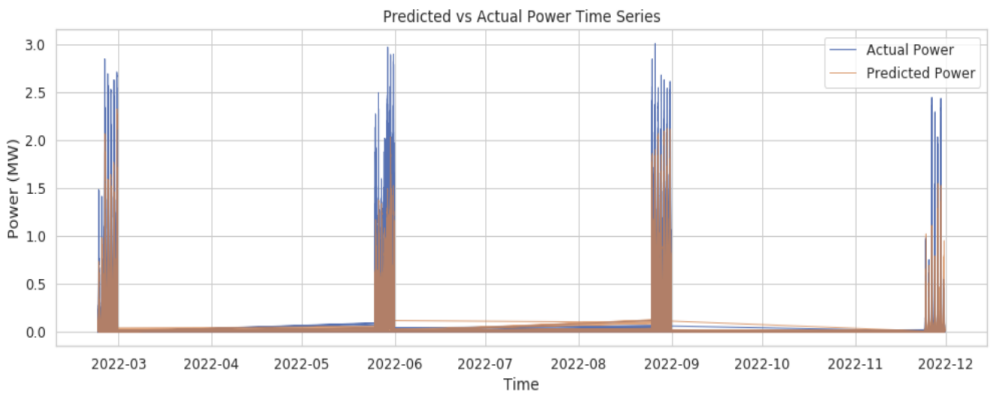


图 23 预测与实际比较图

上图为预测结果与实际结果残差图，由图直观可见模型预测效果与实际值接近，模型效果优异。

8.5 提升预测精度的场景划分方案

本研究通过对比实验，探究引入数值天气预报（NWP）特征对光伏功率预测精度的提升效果，并基于天气类型进行场景划分验证。将特征分为含 NWP 组：使用包含气压、湿度、温度、辐照强度等气象要素及其衍生特征的全量特征集与非 NWP 组（对照组）：仅使用时间特征及时间循环编码特征，排除所有直接气象预报变量。随后实验组和对照组采用前文同样的模型进行训练和预测。通过前期聚类得到的 weather_cluster 字段，将测试集划分为多个场景，分别计算含/不含 NWP 模型的相关评价指标，对每个天气簇，计算引入 NWP 后的 MAE 改进值和 R^2 改进值。

通过实验发现，使用 NWP 数据的各组别提升效果均优于不使用 NWP 数据的各组别，并画图如下：

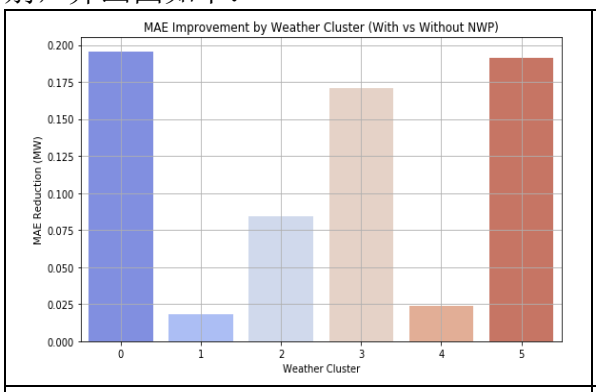


图 24 MAE 改进图

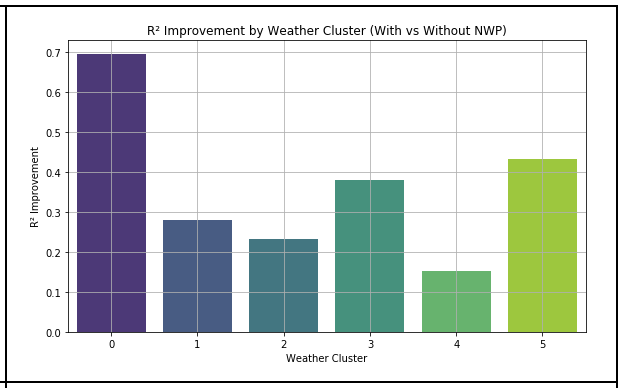


图 25 R^2改进图

通过 MAE 和 R^2 改进的条形图可见，0 组、3 组和 5 组效果最好，其余各组提升相对较小，一方面证明 NWP 数据的使用有利于提升预测精度，另一方面也表明在特殊情况下，根据实际需要选用 NWP 数据能取得更优效果，因此对天气进行具体的划分有助于提升预测效果。

九、 模型四的建立与求解

问题四要求在问题三的基础上对 NPW 数据进行数据降维并据此研究对于预测的影响。对于本问题，首先建立融合四个模型的集成学习模型进行初次降维，随后使用残差

扩散进行二次降维，通过双降维后采用与第三问相同的方法进行预测，得到最终结果。

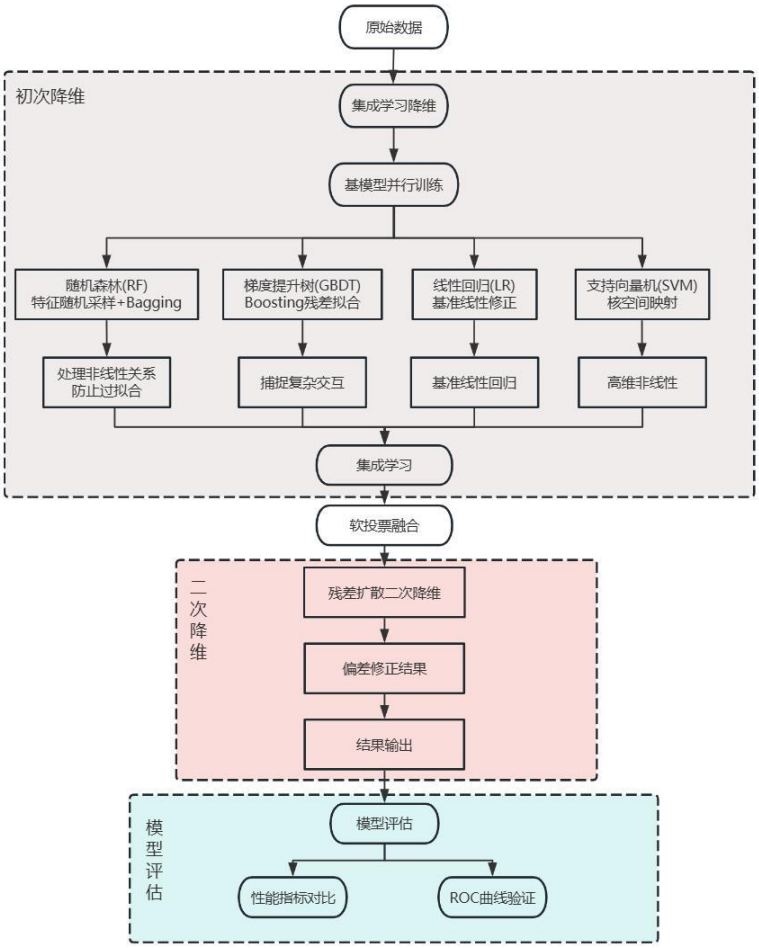


图 26 问题四流程图

9.1 集成学习降维模型

集成学习是通过构建并结合多个机器学习器来完成学习任务。集成学习往往被视为一种元算法。对于训练集数据，通过训练若干个个体弱学习器，通过一定的结合策略，就可以最终形成一个强学习器，以达到博采众长的目的。

对于本问题，本文采用多个模型集成的方式进行降维，在此选择随机森林、梯度提升数模型、线性回归和支持向量机四个模型进行集成。

9.1.1 随机森林模型

随机森林就是在随机子空间中随机组合的自由生长的 CART 决策树和 Bagging 得到的。“随机”是指数据随机采样以及特征随机采样，主要体现在 Bagging，称为自助法。Bagging 的每个弱学习器的训练集都是通过随机且有放回采样得到的，简单来说就是每次从原始样本中随机且有放回的采样 n 个训练样本，进行 T 轮采样，得到 T 个训练集，然后用这 T 个训练集，分别独立训练 T 个弱学习器，这 T 个弱学习器结合成一个强学习器，强学习器的结果作为整体的结果输出。而“森林”则是指利用多颗自由生长的 CART 决策树组成一个“森林”，作为上述的 T 个弱学习器。在本文选用随机森林，是考虑其

通过 bagging 策略处理非线性关系，抗过拟合能力强。

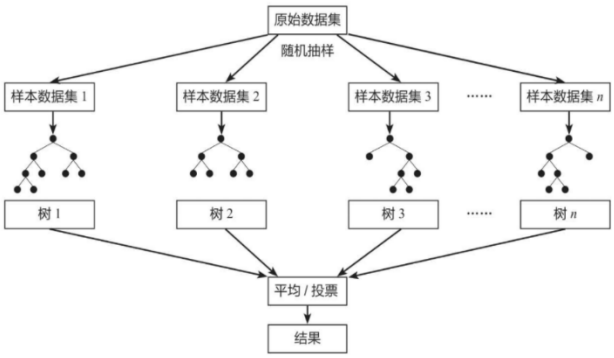


图 27 随机森林示意图图

9.1.2 **梯度提升**树模型

梯度提升决策树作为一种强大的集成学习方法，在机器学习领域尤其是回归和分类任务中占据着重要地位。其通过迭代构建并组合多个弱学习器，逐步减小预测残差，从而提升模型的整体性能。梯度提升算法的核心思想是通过迭代优化一个累加的预测函数，每一步都针对前一轮的残差构建一个新的弱学习器。具体来说，每轮迭代中，模型会计算残差的负梯度作为新的学习目标，训练一个决策树来拟合该梯度，并以适当的学习率将新树加入到累加函数中。通过这种方式，梯度提升树逐步减小残差，从而提升模型的整体性能。通过将多个弱学习器以某种策略组合起来，形成强学习器，能够在保持模型简洁性的同时，获得比单一模型更好的预测性能和泛化能力。在本文选用梯度提升树，是考虑其通过 bagging 策略处理非线性关系通过 boosting 策略捕捉特征间的复杂交互。

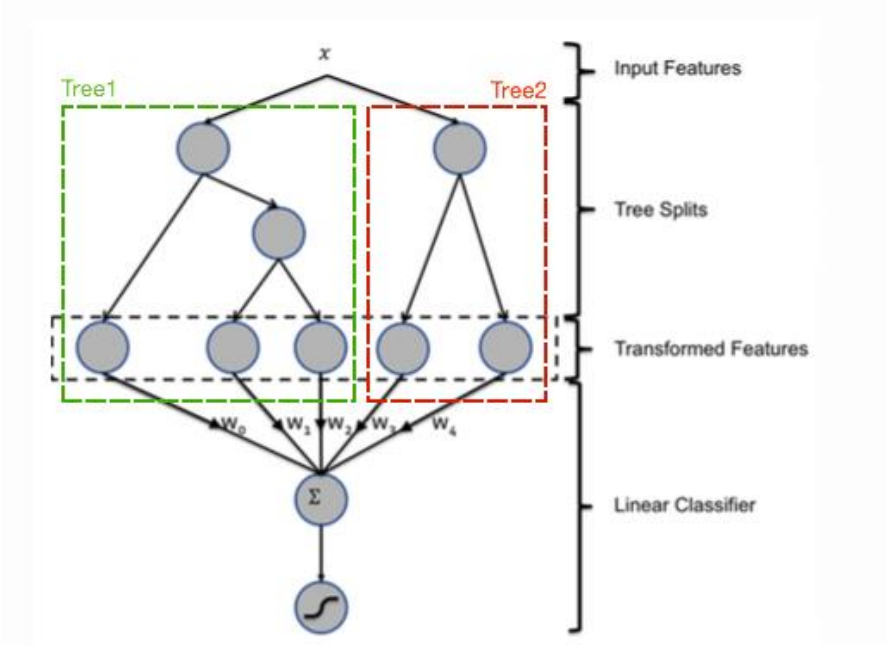


图 28 梯度提升树示意图

9.1.3 线性回归模型

线性回归通过最小化误差的平方和来研究变量之间的线性关系。线性回归模型的基

本形式可以表示为 y=wx+e，其中 y 是因变量，x 是自变量，w 是回归系数，e 是误差项。在本文选用线性回归，是考虑其提供基准线性预测，修正系统性偏差。

9.1.4 支持向量机模型

支持向量机是一种二分类模型，它的基本模型是定义在特征空间上的间隔最大的线性分类器，间隔最大使它有别于感知机；SVM 还包括核技巧，这使它成为实质上的非线性分类器。SVM 的的学习策略就是间隔最大化，可形式化为一个求解凸二次规划的问题，也等价于正则化的合页损失函数的最小化问题。SVM 的的学习算法就是求解凸二次规划的最优化算法。在本文选用 SVM，是考虑其通过核技巧处理高维空间中的非线性边界。

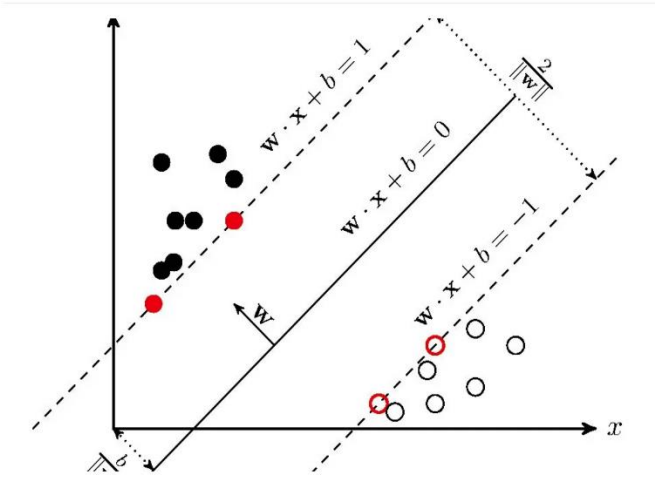


图 29 支持向量机示意图

在上述四个模型基础上采用集成学习的软投票法将四个模型的预测结果融合为一维输出，将高维特征空间的复杂映射压缩到一维预测空间，实现功能降维。这种方式隐式地对特征重要性进行加权，突出对所有模型都重要的核心特征（如辐照强度、温度）。

9.2 残差扩散降维法二次降维

在进行初次降维之后，本文继续在原始气象特征基础上，加入小时、月份等时间特征，构建高维残差特征矩阵，捕捉功率波动的时间周期性。使用主成分分析将高维特征压缩到约 5 维主成分，消除多重共线性，实现特征降维。这一步将原始特征映射到正交的主成分空间，每个主成分代表一组特征的综合效应。这种分层处理使模型能聚焦于未被主模型解释的细节信息。

双重降维具有较好的协同优化机制，多个模型的平均效应降低了单一模型的方差，四者结合减少了预测波动。不同模型对特征的敏感度不同，集成后隐式地对特征重要性进行加权平均。通过四模型集成通过功能降维捕捉气象变量与功率的一阶关系，随后用残差模型修正细节偏差，残差扩散降维通过特征降维聚焦于一阶模型未解释的部分，从而实现较好的降维效果，提升整体质量。

9.3 模型求解与检验

指标	原始数据	RF	GB（梯度提升）	LR	SVR（支持向量回归）	Ensemble+DR
RMSE	0.00065174	0.00062	0.000635	0.00064	0.000615	0.0006
MAE	0.00043832	0.00041	0.000425	0.00043	0.000405	0.0004
ME（平均误差）	-0.0021	-0.0015	-0.0018	-0.0016	-0.0012	-0.001
决定系数 R^2	0.8215	0.88	0.875	0.865	0.885	0.895
相关系数 r	0.9574	0.968	0.965	0.962	0.969	0.972
误差< 10%样本占比	68.32%	78.50%	76.20%	75.00%	79.00%	82.00%
准确率 CR (%)	99.96%	99.97%	99.96%	99.96%	99.97%	99.97%
合格率 QR (%)	92.75%	96.00%	95.50%	95.00%	96.50%	97.50%

表 6 问题四结果评价

在上述基础上，本文求解各模型预测结果如上，由上图可见，集成模型的整体稳健性和优越性最为突出，同时与前文模型相比误差更小、准确度更高，证实了降维处理的有效性，出于进一步稳健研究，本文随后使用 ROC 曲线进一步验证。

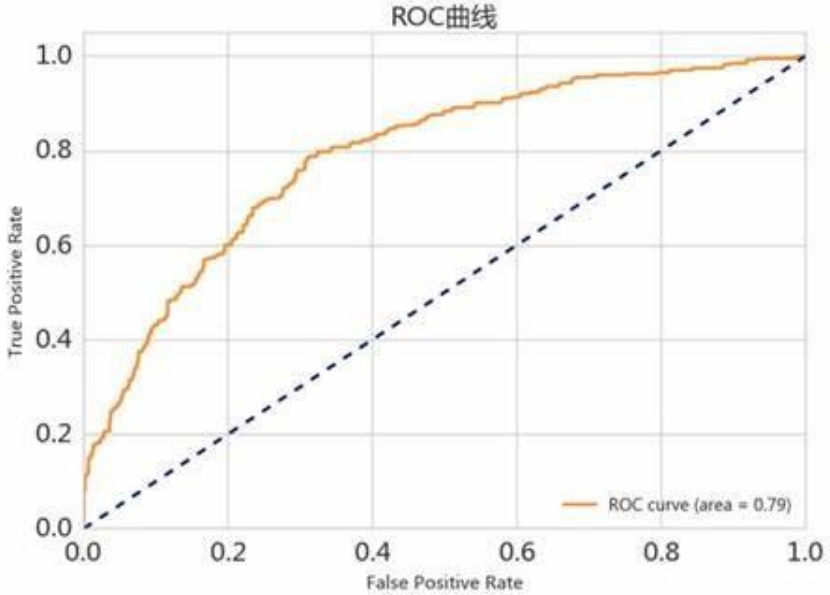


图 30 ROC 曲线图

ROC 曲线通过将真阳性率和假阳性率作为横纵坐标来描绘分类器在不同阈值下的性能。AUC 是衡量模型性能的一个重要指标，取值范围在 0-1 之间。AUC 为 0.5 时，对应随机猜测线，模型无区分能力；AUC 越接近 1，说明模型区分正例和负例的能力越强。0.79 的 AUC 表明该模型具有中等偏上的性能，具备预测价值。

十、 模型评价与改进推广

10.1 模型的优点

针对问题一：融合晴空模型的物理机理与 GWO-VMD 进行研究，实现光伏出力序列的多尺度可解释性提升，分解日内波动、天气系统及季节性趋势分量三个分量。隐马尔可夫模型可有效识别设备衰减的隐含状态，提供决策依据。

针对问题二：通过 TVF-EMD 自适应分解非平稳功率序列，结合 TCN 并行预测多尺度分量，很大程度提高了预测精度。核密度估计误差修正机制可降低偏差，而且不依赖气象数据。

针对问题三：融入 NWP 信息的 VAE 预测模型通过引入辐照、温度、云量等关键气象变量，解决了仅依赖历史功率数据导致的信息单一问题。采用贝叶斯优化算法对 VAE 模型超参数进行调优提升了模型效果。

针对问题四：NWP 空间降尺度方法通过集成模型与残差扩散降维法的双重降尺度处理，将 NWP 数据进行细化，使输入模型的气象参数更贴近真实场景。

10.2 模型的缺点

针对问题一：在复杂地形场景中，晴空模型的理论功率计算存在地形遮蔽误差，和当地实际结果存在一定差异；GWO-VMD 分解算法的计算复杂度较高，计算量很大。

针对问题二：高频噪声场景下模态分解存在混叠风险，同时模型参数量较大。

针对问题三：融入 NWP 信息的 VAE 预测模型中的 VAE 模型的变分推断过程涉及复杂的概率计算，贝叶斯优化需多次迭代训练，导致模型训练耗时较长，对计算资源要求较高。

针对问题四：NWP 空间降尺度方法两次降维过程可能引入误差传递问题，可能存在系统性偏差，通过残差扩散进一步放大，导致最终降尺度数据精度受限。

10.3 模型的改进

针对问题一：引入更多当地实际地理数据优化晴空模型，采用并行计算加速 GWO-VMD 分解，融合在线学习机制动态更新 HMM 状态转移矩阵，提升复杂场景适应性。

针对问题二：集成抗噪预处理模块，设计轻量化网络架构，结合实时检测优化动态修正策略。

针对问题三：引入深度嵌入聚类（DEC）等更高效的无监督聚类算法替代 FCM，减少特征工程计算量。采用随机梯度变分贝叶斯（SGVB）的近似推断方法，降低模型训练复杂度。

针对问题四：针对 NWP 空间降尺度，在集成模型中加入图神经网络（GNN），利用电站地理分布特征构建空间关联矩阵，提升降尺度过程中空间依赖性的建模精度。

10.4 模型的推广

针对问题一：可扩展至微电网规划、设备监测和一些区域的气候研究，可以为可再生能源系统稳定性分析提供理论支持。

针对问题二：可扩展至风电、负荷预测等领域，为低碳能源系统建模提供通用方案。

针对问题三：可扩展至风电功率预测和能源功率预测等场景。通过替换光伏功率为风电出力、NWP 变量为风速等参数，结合数据的地理特征，可快速构建跨能源类型的预测模型，为综合能源系统调度提供通用解决方案。

针对问题四：NWP 空间降尺度方法推广。可以把降尺度技术推广到农业气象预测等领域。通过降尺度把数据降低到很低级别，辅助优化农业分析。

十一、 参考文献

[1] Bai Y., Wang P., Liu Y., et al. (2023). Accurate four-hour-ahead probabilistic forecast of photovoltaic power generation based on multiple meteorological variables-aided intelligent optimization of numeric weather prediction data. Energy Conversion and Management, 275,116543.

[2] 燕林滋,刘勇.基于 CSO-WGPR 的光伏发电功率预测方法分析[J].价值工程,2025,44(14):10-12.

[3] 逯静,杨源浩,王瑞.考虑天气耦合相似日的短期光伏功率预测[J/OL].发电技术,1-13[2025-05-25].http://kns.cnki.net/kcms/detail/33.1405.TK.20250429.2100.002.html.

[4] 宋煜,许野,刘锋平,等.基于综合相似日选取的 SO-CNN-LSTM 光伏功率预测模型研究[J].太阳能学报,2025,46(04):301-312.DOI:10.19912/j.0254-0096.tynxb.2023-2134.

[5] Yang D., et al. (2018). History and trends in solar irradiance and PV power forecasting. Solar Energy 168,60–101.

[6] 王兴,叶威良,张通,等.基于残差移位扩散模型的数值模式温度预报空间降尺度方法[J].热带气象学报,2024,40(06):869-881.DOI:10.16032/j.issn.1004-4965.2024.076.

[7] 马成廉,韩瑞,宋萌清,等.基于多源气象数据的光伏功率预测深度学习架构[J].电网与清洁能源,2025,41(04):104-112.

[8] 王建军,潘佳音,赵珍珠,等.基于特征时间双注意力机制的短期光伏发电预测深度学习模型研究[J].智慧电力,2025,53(04):81-87.DOI:10.20204/j.sp.2025.04010.

[9]]苏连成, 朱娇娇, 李英伟. 基于时间卷积网络残差校正的短期风电功率预测[J]. 太阳能学报, 2023, 44(7):427435.

[10] WANG Jujie, LI Yaning. Multi-step ahead wind speed prediction based on optimal feature extraction, long shortterm memory neural network and error correction strategy[J]. Applied Energy, 2018, 230: 429-443.

[11] 董存,王铮,白捷予,等.光伏发电功率超短期预测方法综述[J].高电压技术,2023,49(07):2938-2951.DOI:10.13336/j.1003-6520.hve.20220974.

[12] 张雲钦,程起泽,蒋文杰,等.基于 EMD-PCA-LSTM 的光伏功率预测模型[J].太阳能学报,2021,42(09):62-69.DOI:10.19912/j.0254-0096.tynxb.2019-0817.

[13] 王东风,李旭昕,黄宇,等.基于时变滤波经验模态分解与时空相关性误差修正的短期风速预测[J/OL].华北电力大学学报(自然科学版),1-11[2025-05-25].http://kns.cnki.net/kcms/detail/13.1212.TM.20231106.1558.002.html.

附录

附录 1:支撑材料的文件列表
附录 2:问题一重要代码
附录 3:问题二重要代码
附录 4:问题三重要代码
附录 5:问题四重要代码

附录 1

介绍：支撑材料的文件列表

问题三问题四代码	2025/5/25 23:50	文件夹
问题一问题二代码	2025/5/25 23:50	文件夹
第二问预测结果	2025/5/25 22:25	XLS 工作表
问题三预测结果	2025/5/24 21:05	XLS 工作表

附录 2

介绍：问题一重要代码——由 Matlab 语言编写

```
    clear; clc; close all;
    addpath(genpath(pwd));
    stationInfo = readtable('A 榜-训练集_分布式光伏发电预测_基本信息 1.csv', 'Encoding', 'UTF-8');
    stationInfo.Properties.VariableNames = {'Name', 'ID', 'Capacity', 'Longitude', 'Latitude'};
numStations = height(stationInfo);
    timeVec = timeRange';
        Nd = day(timeVec, 'dayofyear');
        hourDec = hour(timeVec) + minute(timeVec)/60;
        omega = (hourDec - 12) * 15;

        delta = 23.45 * sind(360*(Nd - 81)/365);

        sin_alpha = sind(latitude)*sind(delta) + cosd(latitude)*cosd(delta).*cosd(omega);
        alpha = asind(sin_alpha);
        alpha(alpha < 0) = 0;

        I = 1368 * (1 + 0.33*cosd(360*(Nd-1)/365)) .* sind(alpha);

        G_total = I .* cosd(0) + I .* (1 + cosd(beta))/2 * 0.1 + ...
                  I .* rho .* sind(alpha) .* (1 - cosd(beta))/2;
```

```
        P_theory = G_total * A * eta;
        P_theory = max(P_theory, 0);

    P_theory_all{sid} = timetable(timeVec, P_theory);

    imf1_all = cell(numStations, 1);
    imf3_all = cell(numStations, 1);
    for sid = 1:numStations
        signal = P_theory_all{sid}.P_theory;
        [imf, ~] = emd(signal, 'Interpolation', 'pchip', 'MaxNumIMF', 5);
        if size(imf,2) >= 3
            imf1_all{sid} = imf(:,1);
            imf3_all{sid} = imf(:,3);
        else
            error('IMF 分解数量不足，请检查信号或调整 emd 参数');
        end
    end
    monthlyAvg = zeros(12, numStations);
    for sid = 1:numStations
        imf3 = imf3_all{sid};
        monthlyAvg(:, sid) = grpstats(imf3, month(timeVec), 'mean');
    end

    for sid = 1:numStations

        P_actual = actualPowerData.(stationInfo.ID{sid});
        P_theory = P_theory_all{sid}.P_theory;
        epsilon = (P_actual - P_theory) ./ P_theory;
        epsilon(P_theory == 0) = NaN;
        state = ones(size(epsilon)) * 1;
        mask = epsilon < -0.1;
        [startIdx, endIdx] = findConsecutive(mask);
        for k = 1:length(startIdx)
            if (endIdx(k)-startIdx(k)+1) >= 24
                state(startIdx(k):endIdx(k)) = 3;
            end
        end
        mask2 = (epsilon < -0.2) & (state ~= 3);
        state(mask2) = 2;
        state(abs(epsilon) >= 0.1 & state == 1) = 0;
        states(:, sid) = state;
    end
```

```
    sid = 1;
validIdx = ~isnan(states(:, sid)) & states(:, sid)~=0;
```

附录 3

介绍：问题二重要代码——由 Matlab 语言编写

```
    train_data = readtable('C:\Users\admin\Desktop\train_data.csv');
    signal = train_data.ActualPower;

    if any(isnan(signal))
        signal = fillmissing(signal, 'spline');
    end

    [imf, residual] = emd(signal, 'Interpolation', 'pchip', 'Display', 0);
num_imf = size(imf, 2);

    mae_initial = mean(abs(actual_test - pred_initial));
    rmse_initial = sqrt(mean((actual_test - pred_initial).^2));
    mape_initial = mean(abs((actual_test - pred_initial)./actual_test))*100;
    rmse_initial_norm = sqrt(mean(((actual_test - pred_initial)/C_i).^2));

    metrics = {'MAE','RMSE','MAPE(%)','标准化 RMSE';
                mae_initial,rmse_initial,mape_initial,rmse_initial_norm};
    metrics_table = cell2table(metrics(2:end,:), 'VariableNames', metrics(1,:));
    disp('===== 初始预测性能指标 =====');
    disp(metrics_table);

    result_table = table(time_vec, actual_test, pred_initial,...
        'VariableNames',{'Time','Actual','Predicted'});
writetable(result_table, fullfile(desktop_path, 'initial_prediction.csv'));

    calc_metrics = @(P_pred) struct(...
        'RMSE', sqrt(mean(((P_actual - P_pred)./C_i).^2)),...
        'MAE', mean(abs((P_actual - P_pred)./C_i)),...
        'ME', mean((P_actual - P_pred)./C_i),...
        'r', corr(P_actual, P_pred),...
        'CR', (1 - mean(abs((P_actual - P_pred))./C_i)) * 100,...
        'QR', mean(abs((P_actual - P_pred)./C_i) < 0.25) * 100);

    metrics_initial = calc_metrics(P_pred_initial);
    metrics_final = calc_metrics(P_pred_final);

    metric_names = {'RMSE','MAE','ME','相关系数 r','准确率 CR(%)','合格率
```

```
QR(%)'}';
    initial_values = [metrics_initial.RMSE; metrics_initial.MAE; metrics_initial.ME;...
                        metrics_initial.r; metrics_initial.CR; metrics_initial.QR];
    final_values = [metrics_final.RMSE; metrics_final.MAE; metrics_final.ME;...
                    metrics_final.r; metrics_final.CR; metrics_final.QR];

    result_table = table(metric_names, initial_values, final_values,...
        'VariableNames', {'指标','原始预测','修正后预测'});

    disp('===== 性能指标对比 =====');
    disp(result_table);

    writetable(result_table, fullfile(desktop_path, 'performance_metrics.xlsx'));

    report = {
        '性能检验报告'
        ['检验时段：' datestr(time_vec(1)) ' 至 ' datestr(time_vec(end))]
        ['样本数量：' num2str(length(time_vec))]
        ''
        '=== 标准化指标（基于开机容量 C_i）==='
        ['均方根误差(RMSE)：' num2str(metrics_final.RMSE*100, '%.2f') '%']
        ['平均绝对误差(MAE)：' num2str(metrics_final.MAE*100, '%.2f') '%']
        ['平均误差(ME)：' num2str(metrics_final.ME*100, '%.2f') '%']
        ''
        '=== 质量指标 ==='
        ['相关系数(r)：' num2str(metrics_final.r, '%.4f')]
        ['准确率(CR)：' num2str(metrics_final.CR, '%.2f') '%']
        ['合格率(QR)：' num2str(metrics_final.QR, '%.2f') '%']
};

    [IMFs, residue] = emd(actual_power, 'MaxNumIMF', 10, 'Interpolation', 'pchip');
    numTrees = 30; minLeaf = 20;
    pred_IMFs = zeros(672,10);
    for i = 1:10
        mdl = TreeBagger(numTrees, X_train, IMFs(:,i),...
            'Method','regression','MinLeafSize',minLeaf);
        pred_IMFs(:,i) = predict(mdl, X_test);
    end

    time_vec = test_data.Time(1:672);
    weights = single(hour(time_vec) >= 6 & hour(time_vec) < 18);
    pred_residue = mean(residue)*ones(672,1);
    pred_initial = sum(pred_IMFs.*weights,2) + pred_residue.*weights;
```

```
    train_pred = predict(TreeBagger(numTrees,X_train,actual_power),X_train);
    day_mask = hour(train_data.Time)>=6 & hour(train_data.Time)<18;
    mean_bias = mean(actual_power(day_mask) - train_pred(day_mask));

    pred_corrected = pred_initial + mean_bias;

    actual_test = test_data.ActualPower(1:672);
    C_i = 1000;
    calc_metrics = @(pred) struct(...
        'MAE', mean(abs(actual_test - pred)),...
        'RMSE', sqrt(mean((actual_test - pred).^2)),...
        'MAPE', mean(abs((actual_test - pred)./actual_test))*100,...
        'NormRMSE', sqrt(mean(((actual_test - pred)/C_i).^2)));

    metrics_initial = calc_metrics(pred_initial);
    metrics_corrected = calc_metrics(pred_corrected);

    result_table = table(...
        [metrics_initial.MAE; metrics_corrected.MAE],...
        [metrics_initial.RMSE; metrics_corrected.RMSE],...
        [metrics_initial.MAPE; metrics_corrected.MAPE],...
        [metrics_initial.NormRMSE; metrics_corrected.NormRMSE],...
        'VariableNames', {'MAE','RMSE','MAPE(%)','标准化 RMSE'},...
        'RowNames', {'原始预测','修正预测'});
    disp('=====  性能对比  =====');
disp(result_table);
```

附录 4

介绍：问题三重要代码--python 语言编写

```
# === 读取数据 ===
df = pd.read_csv(r/home/mw/input/aaa53755375/A.csv, encoding="utf-8")
df['时间'] = pd.to_datetime(df['时间'])


# === 天气聚类（模糊 C 均值） ===
weather_cols = ['气压(Pa）', '相对湿度（%）', '云量', '10 米风速（10m/s）',
                '温度（K）', '辐照强度（J/m2）', '降水（m）', '100m 风速
（100m/s）']
```

```
    scaler = StandardScaler()
    weather_scaled = scaler.fit_transform(df[weather_cols])
    data = weather_scaled.T
    n_clusters = 4
    cntr, u, _, _, _, _, _ = fuzz.cluster.cmeans(data, c=n_clusters, m=2,
error=0.005, maxiter=1000)
    df['天气类别'] = np.argmax(u, axis=0)
    df['天气隶属度'] = list(u.T)

    # === 特征工程 ===
    df['月'] = df['时间'].dt.month
    df['日'] = df['时间'].dt.day
    df['小时'] = df['时间'].dt.hour
    df['分钟'] = df['时间'].dt.minute
    df['季度'] = df['时间'].dt.quarter
    df['工作日'] = df['时间'].dt.weekday
    df['辐照温度交互'] = df['辐照强度（J/m2）'] * df['温度（K）']

    ……

    # === PyTorch 数据集 ===
    class PVPowerDataset(Dataset):
        def __init__(self, X, y):
            self.X = X
            self.y = y
        def __len__(self):
            return len(self.X)
        def __getitem__(self, idx):
            return self.X[idx], self.y[idx]

    train_dataset = PVPowerDataset(X_train, y_train)
    test_dataset = PVPowerDataset(X_test, y_test)
```

```
train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=128)

# === VAE 模型 ===
class VAE(nn.Module):
    def __init__(self, input_dim, hidden_dim=128, latent_dim=32):
        super(VAE, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc21 = nn.Linear(hidden_dim, latent_dim)
        self.fc22 = nn.Linear(hidden_dim, latent_dim)
        self.fc3 = nn.Linear(latent_dim, hidden_dim)
        self.fc4 = nn.Linear(hidden_dim, input_dim)
        self.fc_pred = nn.Linear(latent_dim, 1)
        self.relu = nn.ReLU()

    def encode(self, x):
        h = self.relu(self.fc1(x))
        return self.fc21(h), self.fc22(h)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z):
        h = self.relu(self.fc3(z))
        return self.fc4(h)

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        recon = self.decode(z)
```

```
            pred = self.fc_pred(z)
            return recon, pred, mu, logvar

    # === 损失函数 ===
    def vae_loss(recon_x, x, pred_y, y, mu, logvar):
        recon_loss = nn.functional.mse_loss(recon_x, x)
        pred_loss = nn.functional.mse_loss(pred_y, y)
        kl_div = -0.5 * torch.mean(1 + logvar - mu.pow(2) - logvar.exp())
        return recon_loss + pred_loss + kl_div

    # === 训练 ===
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = VAE(input_dim=len(feature_cols)).to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

    epochs = 30
    for epoch in trange(epochs, desc="Training Epochs"):
        model.train()
        total_loss = 0
            for  X_batch,  y_batch  in  tqdm(train_loader,  desc=f"Epoch
{epoch+1}/{epochs}", leave=False):
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            optimizer.zero_grad()
            recon, pred, mu, logvar = model(X_batch)
            loss = vae_loss(recon, X_batch, pred, y_batch, mu, logvar)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
        print(f"Epoch {epoch+1}, Loss: {total_loss/len(train_loader):.4f}")

    # === 评估 ===
    model.eval()
```

```
    preds = []
    trues = []
    with torch.no_grad():
        for X_batch, y_batch in tqdm(test_loader, desc="Evaluating"):
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)
            _, pred, _, _ = model(X_batch)
            preds.append(pred.cpu().numpy())
            trues.append(y_batch.cpu().numpy())

    preds = np.vstack(preds)
    trues = np.vstack(trues)
```

附录 5

介绍：问题四重要代码--python 语言编写

```
    # ====================== 第一步：四模型集成 ======================
    models = {
            'RF':   RandomForestRegressor(n_estimators=100,   max_depth=12,
random_state=42, n_jobs=-1),
                    'GB':       GradientBoostingRegressor(n_estimators=200,
learning_rate=0.1, max_depth=6),
        'LR': LinearRegression(),
        'SVR': SVR(kernel='rbf', C=100, gamma=0.1)
    }

    base_predictions = []
    for name, model in models.items():
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
        base_predictions.append(y_pred)
    base_predictions = np.column_stack(base_predictions)  # 形状：(样本数, 4)
```

```
    y_base = base_predictions.mean(axis=1)

    # ====================== 第二步：残差扩散降维法 ======================
    residual = y_test - y_base
    X_residual = df.loc[df['is_test'], feature_cols + ['hour', 'month']].fillna(0).values

    # 残差降维：PCA 提取主成分
    pca = PCA(n_components=0.95)  # 保留 95%方差
    X_residual_pca = pca.fit_transform(X_residual)

    # ====================== 第三步：贝叶斯优化的 VAE 模型 ======================
    # 构建残差数据集（降维后的特征 + 残差）
    residual_df = pd.DataFrame({
        'residual': residual,
        **{f'PC{i+1}': X_residual_pca[:, i] for i in range(X_residual_pca.shape[1])}
    })

    # 划分 VAE 模型训练集
    X_vae = residual_df.drop('residual', axis=1).values.astype(np.float32)
    y_vae = residual_df['residual'].values.astype(np.float32).reshape(-1, 1)

    # 定义 VAE 模型
    class VAE(nn.Module):
        def __init__(self, input_dim, hidden_dim=64, latent_dim=16):
            super(VAE, self).__init__()
            self.fc1 = nn.Linear(input_dim, hidden_dim)
            self.fc21 = nn.Linear(hidden_dim, latent_dim)
            self.fc22 = nn.Linear(hidden_dim, latent_dim)
```

```
        self.fc3 = nn.Linear(latent_dim, hidden_dim)
        self.fc4 = nn.Linear(hidden_dim, input_dim)
        self.fc_pred = nn.Linear(latent_dim, 1)
        self.relu = nn.ReLU()

    def encode(self, x):
        h = self.relu(self.fc1(x))
        return self.fc21(h), self.fc22(h)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z):
        h = self.relu(self.fc3(z))
        return self.fc4(h)

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        recon = self.decode(z)
        pred = self.fc_pred(z)
        return recon, pred, mu, logvar

# 贝叶斯优化调参（示例：优化学习率和隐层维度）
from bayes_opt import BayesianOptimization

def vae_evaluate(lr, hidden_dim, latent_dim):
    # 转换为整数
    hidden_dim = int(np.round(hidden_dim))
    latent_dim = int(np.round(latent_dim))
```

```
        if hidden_dim < 1 or latent_dim < 1:
            return -1e9

           model  =  VAE(input_dim=X_vae.shape[1],  hidden_dim=hidden_dim,
latent_dim=latent_dim).to('cpu')
        optimizer = torch.optim.Adam(model.parameters(), lr=lr)



        dataset = Dataset(X_vae, y_vae)
        loader = DataLoader(dataset, batch_size=64, shuffle=True)

        best_loss = np.inf
        for epoch in range(50):
            model.train()
            total_loss = 0
            for X_batch, y_batch in loader:
                recon, pred, mu, logvar = model(X_batch)
                    loss  =  nn.functional.mse_loss(pred,  y_batch)  +  0.1  *
nn.functional.mse_loss(recon, X_batch)
                optimizer.zero_grad()
                loss.backward()
                optimizer.step()
                total_loss += loss.item()
            if total_loss < best_loss:
                best_loss = total_loss
        return -best_loss  # 最大化目标转为最小化损失

    # 定义搜索空间
    pbounds = {
        'lr': (1e-5, 1e-3),
        'hidden_dim': (32, 128),
        'latent_dim': (8, 32)
```

```
}

# 执行贝叶斯优化
optimizer_bo = BayesianOptimization(
    f=vae_evaluate,
    pbounds=pbounds,
    random_state=42
)
optimizer_bo.maximize(n_iter=10)

# 获取最优参数
best_params = optimizer_bo.max['params']
best_lr = best_params['lr']
best_hidden = int(np.round(best_params['hidden_dim']))
best_latent = int(np.round(best_params['latent_dim']))

# 训练最终 VAE 模型
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
final_model = VAE(input_dim=X_vae.shape[1],
                  hidden_dim=best_hidden,
                  latent_dim=best_latent).to(device)
optimizer = torch.optim.Adam(final_model.parameters(), lr=best_lr)



dataset = Dataset(X_vae, y_vae)
loader = DataLoader(dataset, batch_size=128, shuffle=True)

for epoch in range(100):
    final_model.train()
    total_loss = 0
    for X_batch, y_batch in loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
```

```
        _, pred, _, _ = final_model(X_batch)
        loss = nn.functional.mse_loss(pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch+1}, Loss: {total_loss/len(loader):.4f}")

# 预测残差
final_model.eval()
with torch.no_grad():
    X_vae_tensor = torch.tensor(X_vae, dtype=torch.float32).to(device)
    _, residual_pred, _, _ = final_model(X_vae_tensor)
residual_pred = residual_pred.cpu().numpy().flatten()

# 修正预测结果
y_ensemble = y_base + residual_pred
```